# MySQL / MariaDB VS PostgreSQL

Cara a Cara Arquitectónico: Concurrencia, Indexación, Motores de Ejecución, HA y Casos de Uso

RDBMS ARCHITECTURE CHEATSHEET

| Vector de Análisis | 🐬 MySQL / MariaDB (InnoDB Engine) | 🐘 PostgreSQL (ORDBMS Core) |
|---|---|---|
| **Modelo Concurrencia & MVCC** | • Usa **Undo Logs** & Buffer Pool.<br>• In-place updates (sobrescribe en la misma página de memoria/disco). No sufre de bloat dinámico masivo de tablas.<br>• Lock Manager por registro y rangos (Next-Key Locking para prevenir Phantom Reads). | • MVCC basado en **Append-only (Tuple Copy)**.<br>• Cada UPDATE/DELETE escribe una nueva versión completa de la tupla.<br>• **Requiere Mantenimiento** Obliga al proceso `VACUUM` (Auto-Vacuum) a limpiar 'dead tuples' y prevenir bloat. |
| **Tipos de Datos & Extensiones** | • Soporte JSON/JSONB nativo con operadores `->` / `->>` y columnas generadas virtualmente.<br>• Tipos espaciales GIS básicos (SRID).<br>• **Ecosistema Cerrado:** No permite extender internamente el motor con nuevos tipos o índices sin compilar C++. | • **Ganador en Flexibilidad** **PostGIS** (estándar global de GIS).<br>• Soporte completo de `JSONB` indexable mediante GIN/GiST.<br>• Soporte vectorial para IA / LLMs con **pgvector**.<br>• Arrays nativos, Enums, Custom Types y HSTORE. |
| **Estrategias de Indexación** | • Enfocado en **B-Tree** y R-Tree.<br>• Tabla organizada en torno a la Primary Key (Clustered Index nativo e imperativo).<br>• Full-text search integrado para InnoDB. | • **Soporte Multialgoritmo** **B-Tree, Hash, GIN, GiST, SP-GiST, BRIN**.<br>• **BRIN:** Ultra-eficiente en tablas masivas con orden cronológico (time-series).<br>• **Partial Indexes:** `WHERE status = 'ACTIVE'`.<br>• **Functional Indexes:** Índices sobre expresiones SQL. |
| **Ejecución de Queries & Joins** | • Históricamente basado en **Nested Loop Join**.<br>• MySQL 8.0 introdujo **Hash Join** (optimizado para consultas analíticas básicas).<br>• **Single-threaded:** Ejecuta cada query dentro de un único hilo de CPU. | • **Motor Avanzado** Hash Join, Merge Join y Nested Loop Join dinámicos.<br>• **Parallel Query Execution:** Divide escaneos y joins en múltiples cores de CPU por query.<br>• Planificador de consultas ultra sofisticado orientado a costo (CBO). |
| **Replicación & Alta Disponibilidad** | • **Binlog Replication** (Row-based, Statement, Mixed).<br>• Replicación Asíncrona, Semi-síncrona y **Group Replication / InnoDB Cluster**.<br>• Muy maduro para escalabilidad horizontal de lectura con ProxySQL. | • **WAL Streaming Replication** a nivel de bloque (Física).<br>• **Logical Replication:** Basada en modelo Pub/Sub por tablas.<br>• Herramientas de HA robustas: Patroni, PgBouncer, Stolon. |
| **Modelo de Procesos & Conexiones** | • Modelo **Thread-per-connection**.<br>• Ligero y eficiente al manejar miles de conexiones simultáneas con bajo consumo de memoria base. | • Modelo **Process-per-connection** (vía `fork`).<br>• **Requiere Pooler** Cada conexión consume más recursos de RAM/OS. Requiere obligatoriamente un Connection Pooler (PgBouncer / Supavisor). |
| **Casos de Uso Ideales** | • Cargas OLTP puras con alto volumen de lecturas y escrituras simples.<br>• Plataformas Web, CMS (WordPress, Drupal) y E-commerce.<br>• Aplicaciones donde la velocidad en consultas directas por PK es prioritaria. | • Sistemas Financieros, Datos Geoespaciales e IA/Embeddings.<br>• Cargas Mixtas OLTP + OLAP (HTAP) y reporteo complejo.<br>• Proyectos con esquemas JSON complejos y reglas de integridad estrictas. |